

REPORT

Intelligent Urban Parking Space Optimization

SUBMITTED BY:

S.No.	Name
1	Sarthak Gupta

TABLE OF CONTENTS

Chapter No.	Topics	Page No.
	List of Figures	
Chapter-1	Introduction 1.1 Abstract 1.2 Introduction 1.3 Objectives 1.4 Scope of the Project	8-11
Chapter-2	Literature Review 2.1 Review of existing parking systems 2.2 Review of Journal and Conference Papers 2.3 Identified Research Gaps	12-13
Chapter-3	System Design 3.1 Block Diagram 3.2 Block Diagram Explanation	14-15
Chapter-4	Components Used 4.1 Arduino Uno 4.2 IR Sensors 4.3 LCD Display (16x2 with I2C Module) 4.4 Servo Motor (SG90) 4.5 Breadboard 4.6 Connecting Wires 4.7 Power Supply	16-21

Chapter 5	Circuit Implementation	22-24
	5.1 Circuit Diagram	
	5.2 Circuit Explanation	
Chapter 6	Working of the system	25-26
	6.1 Step-by-Step Working	
	6.2 System Operation Summary	
Chapter 7	Logic Design	27-28
	7.1 Flowchart	
	7.2 Algorithm	
Chapter 8	Software and Programming	29-34
	8.1 Software Used (Arduino IDE)	
	8.2 Required Libraries	
	8.3 Arduino Code	
	8.4 Code Explanation	
Chapter 9	Discussion	35-37
	9.1 Advantages of the system	
	9.2 Applications	
	9.3 Limitations of the system	
Chapter 10	Results and Analysis	38-42
	10.1 Results and Observations	
	10.2 Testing and Analysis	
Chapter-11	Future Scope	43
Chapter-12	Conclusion	44

LIST OF FIGURES

FIG NO.	FIGURE TITLE	PAGE NO.
Fig 3.1	Block Diagram of the Project	14
Fig 4.1	Arduino Uno	16
Fig 4.2	IR Sensor	17
Fig 4.3	Servo Motor (SG90)	18
Fig 4.4	16x2 LCD display with I2C Module	19
Fig 4.5	USB Cable	20
Fig 4.6	Jumper Wires (Male to Male, Male to Female, Female to Female)	21
Fig 4.7	Breadboard	21
Fig 5.1	Circuit Diagram	22
Fig 7.1	Flowchart of the project	27
Fig 8.1	Arduino IDE Workspace	30
Fig 10.1	Circuit Explanation	39
Fig 10.2	LCD Display Successful	39
Fig 10.3(a) Fig 10.3(b) Fig 10.3(c)	Arduino Code Successful	40-41

CHAPTER - 1

ABSTRACT

This project presents a simplified yet powerful implementation of a Intelligent Urban Parking Space Optimization using Arduino Uno, two IR sensors, a servo motor, and a 16×2 LCD display. The major objective of this system is to automate vehicle entry and exit detection and display real-time parking slot availability.

The proposed system uses two IR sensors to detect vehicles at the entry and exit of a parking area. Based on the sensor inputs, the system calculates available parking slots and displays them on an LCD display. A servo motor acts as an automated gate that opens or closes according to parking availability.

This project is highly efficient, low-cost, reliable, and suitable for small-scale parking applications like:

- Apartment parking
- Office building parking
- Mall entry systems
- Project & lab demonstrations

The report explains hardware components, circuit design, working principle, Arduino programming, system flowcharts, advantages, limitations, cost analysis, and future scope. The project can be easily upgraded into a fully IoT-enabled parking system using Wi-Fi and cloud monitoring.

INTRODUCTION

Parking management has become an essential requirement due to the rapidly increasing number of vehicles in urban areas. Traditional parking systems involve manual checking, human guards, and guess-based slot availability, which often lead to congestion, time waste, and mismanagement.

To solve these challenges, smart automated systems have been developed. An Intelligent Urban Parking Space Optimization helps drivers by:

- Automatically detecting free slots
- Displaying real-time availability
- Reducing manual intervention
- Improving time efficiency
- Enhancing safety and convenience

This project aims to design a simple, low-cost, prototype-level Intelligent Urban Parking Space Optimization using basic sensors and microcontrollers. Arduino Uno serves as the brain of the system, while IR sensors detect vehicles entering or exiting the parking lot. A servo motor controls a physical gate based on slot availability. The 16×2 LCD displays the number of remaining slots.

The system works without Wi-Fi, servers, or cloud — making it perfect for learning embedded systems basics, real-time sensing, and automation.

Intelligent Urban Parking Space Optimization are crucial installations in:

- Offices
- Malls
- Public parking lots
- Apartments
- Campuses

This project introduces to:

- Real-time embedded systems
- Sensor interfacing
- Display systems
- Mechatronics (servo-based actuation)
- System automation

The goal of this system is to present a working prototype where:

- Vehicles are detected using IR sensors
- Slot count updates automatically
- The gate operates using a servo
- Users see live updates on an LCD

OBJECTIVES

1. To design a compact and efficient smart parking model.
2. To automate vehicle entry and exit using IR sensors.
3. To control the barrier gate through a servo motor.
4. To display real-time parking slot availability using LCD.
5. To reduce human involvement in the parking system.
6. To implement a simple, cost-effective embedded system.
7. To create a scalable prototype for future IoT expansion.

SCOPE OF THIS PROJECT – NATIONAL

The scope of this project is **National**, as the Intelligent Urban Parking Space Optimization directly addresses one of India's most critical and rapidly growing challenges — the problem of disorganized and inefficient parking management. With the increasing population, rising number of vehicles, expansion of commercial spaces, and ongoing urban development, Indian cities are experiencing severe traffic congestion and parking shortages.

India is currently witnessing a massive push toward modernization under the **Smart Cities Mission**, which aims to integrate technology-driven solutions into urban infrastructure. Parking remains an essential part of this mission because it affects mobility, traffic flow, fuel consumption, and overall citizen convenience. A smart, automated, and low-cost parking solution like this project aligns perfectly with the national agenda of developing **sustainable, digital, and intelligent transportation systems**.

This project can be implemented at the national level in a wide variety of Indian environments, such as:

- Metro cities including **Delhi, Mumbai, Bengaluru, Kolkata, Chennai, and Hyderabad**, where multi-level parking and automated systems are urgently needed.
- Tier-2 and Tier-3 cities that are growing rapidly and require cost-effective solutions to manage increasing vehicle loads, such as **Pune, Ahmedabad, Lucknow, Chandigarh, Jaipur, Indore, and Kochi**.
- Public infrastructure locations such as **railway stations, airports, metro stations, government buildings, hospitals, and educational institutions**, where organized parking plays a vital role in crowd control.
- Commercial spaces including **malls, IT parks, market complexes, theatres, and hotels**, where high traffic flow demands automated entry and exit systems.
- Residential areas and housing societies across India that require structured and secure parking solutions.

The system's use of inexpensive components such as Arduino, IR sensors, and servo motors makes it ideal for widespread deployment across the country. Unlike advanced parking technologies that require expensive hardware or complex networking, this model is **simple, affordable, and suitable for Indian economic conditions**, particularly in semi-urban and developing regions.

Furthermore, the system supports India's national goals in the following ways:

- **Digital India & Automation:** Encourages adoption of small-scale automation in daily life.
- **Smart Mobility:** Helps build smart parking modules as part of Intelligent Transportation Systems (ITS).
- **Energy & Cost Efficiency:** Reduces manpower and minimizes operational costs.
- **Sustainable Development:** Reduces fuel wastage caused by searching for parking spots.
- **Scalability & Expansion:** The system can be upgraded easily with IoT, cloud monitoring, and payment integration for deployment across large public parking zones.

In summary, the **national scope** of this project is vast and highly relevant. The prototype serves as a foundational model for developing large-scale automated parking systems that India urgently needs to support its rapidly growing urban centres. This project not only demonstrates technological innovation

but also supports India's broader vision of developing **smart, efficient, affordable, and future-ready urban infrastructure**.

CHAPTER – 2

LITERATURE REVIEW

The concept of Intelligent Urban Parking Space Optimization has gained significant attention in recent years due to the rise in urbanization, vehicle population, and the need for efficient mobility solutions. Traditional parking methods in many cities rely on manual supervision, paper-based ticketing, human-operated barriers, and guess-based parking slot availability, which often lead to congestion, time waste, and operational inefficiency. Researchers and institutions worldwide have proposed various technological models to automate parking processes using embedded systems, sensor networks, and IoT-based solutions.

One of the notable studies in this field was conducted by **Al-Humaidi and Mahmood (2021)**, who developed an Arduino-based smart parking system that utilized IR obstacle sensors for detecting vehicle entry and exit. Their research emphasized the feasibility of building cost-effective automated parking systems using commonly available components. The authors demonstrated that microcontrollers such as Arduino Uno are capable of handling real-time detection, logical decision-making, and control output systems such as servo or relay-based gates. Their experimental results validated that IR sensors, despite being low-cost, provide reliable detection within short distances, making them suitable for small commercial complexes, housing societies, and institutional campuses. This study forms the foundation for simplified prototypes like the one developed in the present project.

Expanding upon this, **Bharamagoudar, Patil, and Sasnur (2019)** proposed an IoT-enabled parking system tailored for smart city applications. Their research, presented in an international smart city conference, highlighted the importance of integrating cloud platforms and wireless networks with sensor-based parking modules. They introduced a multi-layered architecture consisting of sensing units, communication modules, cloud servers, and an Android application interface. Their findings showed that real-time parking data, when made accessible via mobile apps, boosts user convenience and reduces the overall traffic load caused by drivers searching for free parking spaces. Although more advanced than Arduino-based modules, the system's core sensing mechanism is still dependent on IR or ultrasonic sensors, confirming the relevance of sensor-based prototypes like the one used in this project.

Another key contribution to sensing methodologies was provided by **Singh and Gupta (2019)** in their IEEE Sensors Journal publication. Their study analyzed the performance of infrared sensors for real-time vehicle detection under different lighting conditions, distances, and surface reflectivity. The authors concluded that IR sensors provide quick response times (less than 100 ms), low power consumption, and high reliability when used within their optimal range. They also discussed environmental factors such as sunlight interference, which can affect detection accuracy outdoors but have minimal effect indoors or in controlled environments. Their experimental curves and signal analysis validated the suitability of IR sensors for short-range automated systems, including automated gates, robotic sensing, industrial counters, and smart parking modules.

In addition to these studies, the field of embedded systems has also evolved significantly with advancements in microcontrollers and efficient programming environments. Arduino, being an open-source platform with a large community ecosystem, has been widely used in many academic and industrial projects. According to the official Arduino documentation (2024), the Arduino Uno based on ATmega328P offers robust digital and analog I/O capabilities suitable for real-time sensor interfacing, automation control, and small-scale mechatronics applications. This platform provides stable PWM output channels required for precise servo motor control, which is an essential part of automated parking barrier systems.

When analyzing previous literature, it becomes clear that the majority of smart parking systems can be grouped into four major categories:

1. **Infrared Sensor-Based Systems** – Fast, low-cost, beginner-friendly systems suitable for small areas.
2. **Ultrasonic Sensor-Based Systems** – More accurate but expensive; suited for large-scale deployments.
3. **RFID-Based Systems** – Useful for vehicle identification and paid parking models.
4. **IoT and Cloud-Based Systems** – Provide online slot information and mobile app integration.

The prototype developed in this project belongs primarily to the first category, using **infrared sensors** for vehicle detection due to their affordability, ease of use, and adequate accuracy for small-scale parking.

Comparing the above literature, it is evident that many of the advanced IoT systems are derived from simple embedded prototypes. The Arduino + IR sensor architecture used in this project matches the base-level sensing and control approach described in the studies of Al-Humaidi & Mahmood (2021) and Singh & Gupta (2019). While these prior works focus either on full system implementation or sensor evaluation, the present project combines both approaches by implementing a functional prototype that automates gate control, monitors parking slots, and displays real-time information on an LCD.

The literature also highlights a major technological challenge — environmental noise. IR sensors can occasionally suffer from false triggers due to sunlight or reflective surfaces. However, this project is intended for **indoor or controlled environments**, making IR sensors a cost-effective and reliable choice. This is consistent with the findings of the IEEE Sensors Journal paper.

Furthermore, the IoT-oriented study by Bharamagoudar et al. (2019) establishes the long-term scalability of such systems. Even though the current project uses only Arduino and non-IoT components, the framework is easily extendable to Wi-Fi modules such as ESP8266/ESP32, enabling cloud-based monitoring, mobile apps, and dashboard visualizations—features that are nationally and globally adopted in modern smart cities.

In conclusion, the literature strongly supports the relevance, feasibility, and scalability of Arduino-based smart parking systems. Prior research demonstrates that IR sensors are reliable for vehicle detection, Arduino provides efficient control and processing, and embedded prototypes like this serve as the groundwork for national-level smart city infrastructure development. Thus, this project aligns with both academic research trends and the practical needs of modern urban India.

CHAPTER – 3

BLOCK DIAGRAM

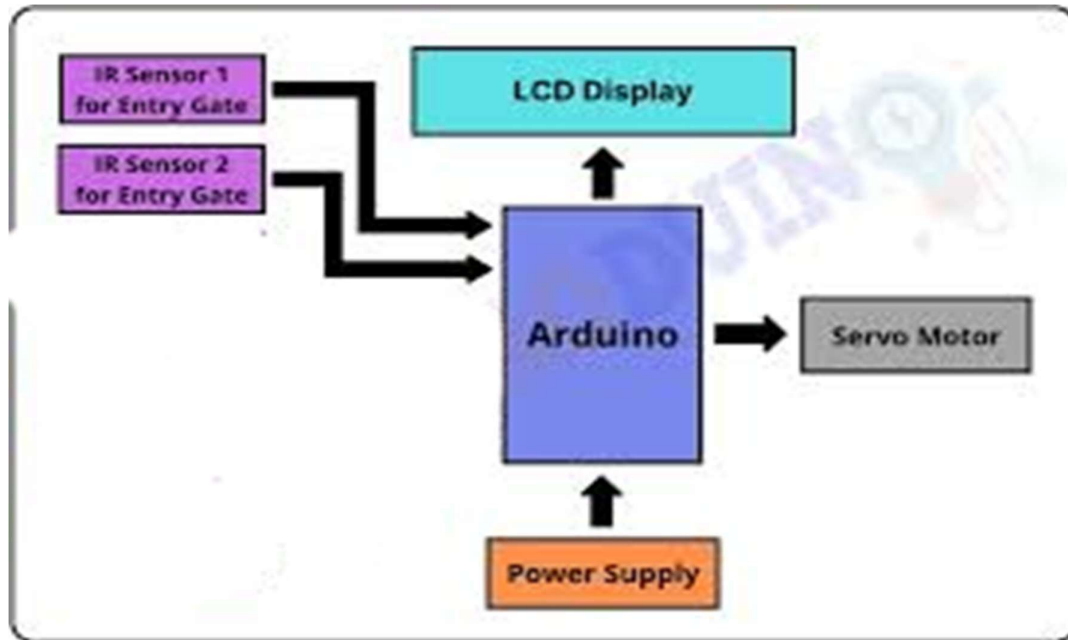


Fig. 3.1 Block Diagram of the project

Block Diagram Overview

An Intelligent Urban Parking Space Optimization prototype consists of interconnected hardware modules that work together to detect vehicles, count available slots, automate gate movement, and display results. The simplified block diagram includes:

1. IR Sensor 1 (Entry Detection)
Detects when a vehicle arrives at the entry point. Sends signal to Arduino.
2. IR Sensor 2 (Exit Detection)
Detects when a vehicle leaves. Helps increment or decrement slot count.
3. Arduino Uno
Acts as the brain of the system. Reads sensor values, updates slot count, and controls servo gate + LCD.
4. Servo Motor (Gate Control)
Opens the gate when parking slots are available.
Closes the gate when the parking is full.

5. 16×2 LCD Display (I2C)
Shows the number of available parking slots.
6. Power Supply
USB or 5V regulated supply to power all components.

Detailed Explanation of Data Flow

Step 1: Vehicle Approaches → IR1 Triggered
IR1 detects an object by sensing the interruption in IR light.
Arduino receives the signal through digital pin.

Step 2: Arduino Performs Logic Calculation
If slot > 0 → gate opens, slot decreases.
Else → display “Parking Full”.

Step 3: Vehicle Entry Complete → IR2 Triggered
IR2 confirms vehicle has passed the gate region.

Step 4: Servo Motor Rotation
Servo moves between:

- 0° (Gate Open)
- 90–100° (Gate Closed)

Step 5: LCD Updates Slot Count
Shows:

Slot Left: X

Functional Advantages of the Block Diagram

- Clear separation of input (IR sensors), processing (Arduino), and output (Servo + LCD).
- Easy to expand (add more sensors or convert to IoT system).
- Beginner-friendly wiring system.

CHAPTER – 4

COMPONENTS USED

1. Arduino Uno (ATmega328p)

Arduino Uno is an open-source microcontroller board based on the ATmega328P chipset. It is widely used for academic, industrial, and hobby electronics projects. It contains:

- 14 digital I/O pins
- 6 analog input pins
- 16 MHz crystal oscillator
- USB interface
- On-board voltage regulator
- Reset switch

Why Arduino is used in this project?

- Easy to program (Arduino IDE)
- Supports C/C++ language
- Inexpensive and reliable
- Can interface with sensors, motors, displays easily
- Real-time processing capability

Role in this project

Arduino Uno reads IR sensor input values, calculates slot availability, and controls the servo motor. It also sends data to the LCD for display.

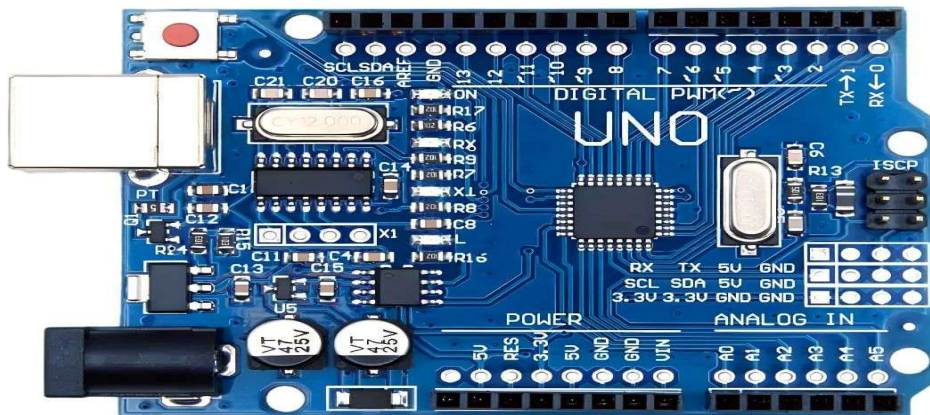


Fig. 4.1 Arduino Uno

2. IR Sensor (Infrared Obstacle Detection Sensor)

IR sensors detect obstacles by transmitting IR light and receiving the reflected signal. They consist of:

- IR transmitter (LED)
- IR receiver (photodiode)
- Comparator (LM393)
- Output LED indicator
- Sensitivity control potentiometer

Working Principle

- When an object blocks IR light → sensor output changes (HIGH or LOW depending on module type).
- Arduino detects this change and updates slot count.

Why IR sensors are used?

- Low cost
- Easy to interface
- High responsiveness
- Suitable for short-range detection

Role in this project

- IR1 detects vehicle entry
- IR2 detects vehicle exit



Fig. 4.2 IR Sensor

3. Servo Motor (SG90)

Servo motor rotates between 0° and 180° using PWM signals.

Features:

- Operating voltage: 4.8V–5V
- Torque: 1.8–2 kg/cm
- Gearbox inside
- Shaft movement based on PWM pulses

Why Servo Motor?

- Accurate angle control
- Lightweight
- Low power consumption
- Ideal for mini mechanical gates

Role in project:

- Acts as gate barrier
- Opens when vehicle is allowed
- Closes when slots = full



Fig. 4.3 Servo Motor (SG90)

4. 16×2 LCD Display with I2C Module

LCD is used to show parking slot count clearly.

Advantages of I2C Module:

- Reduces wiring from 16 pins to only 4 pins
- Uses SDA and SCL lines
- Easy to program using LiquidCrystal_I2C library

Role in project:

Displays messages like:

```
WELCOME!  
Slot Left: 4
```

and

```
SORRY :(  
Parking Full
```

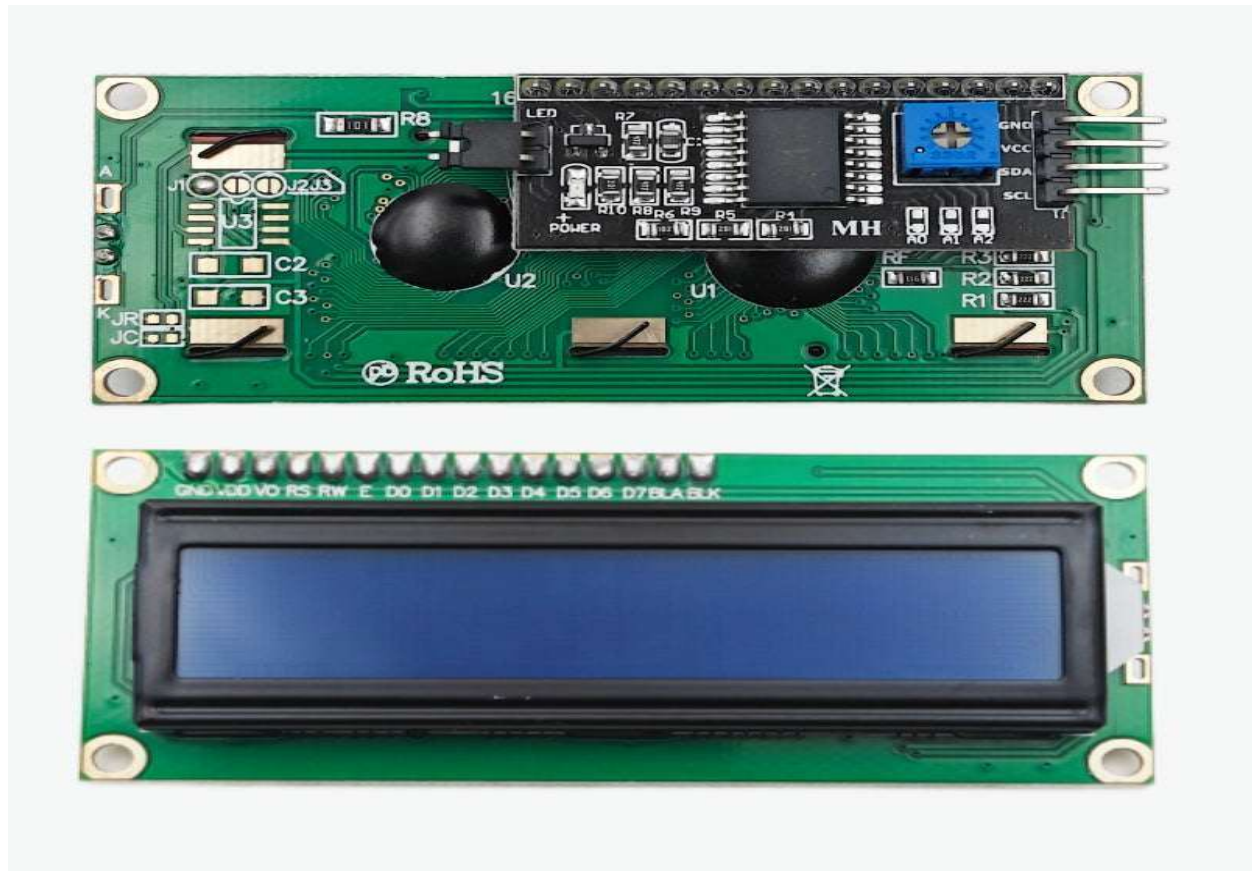


Fig. 4.4 16x2 LCD display with I2C Module

5. Power Supply (5V USB or Adapter)

Arduino and sensors require regulated 5V.
Servo motor also works on 5V.

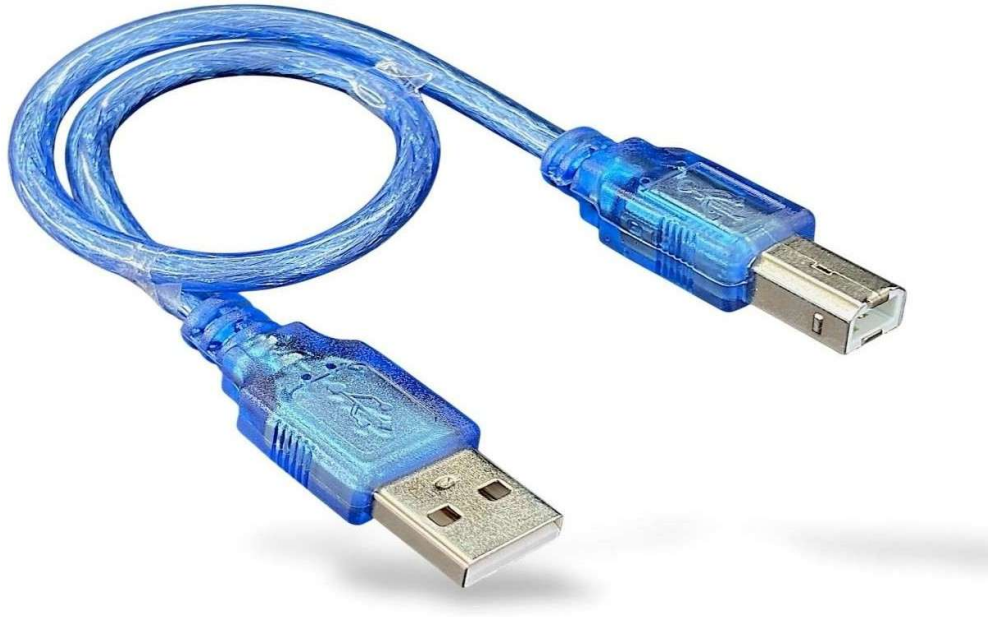


Fig. 4.5 USB Cable

6. Connecting Wires (Male-to-Male, Male-to-Female)

Used to connect:

- Sensors
- LCD
- Servo
- Arduino I/O pins

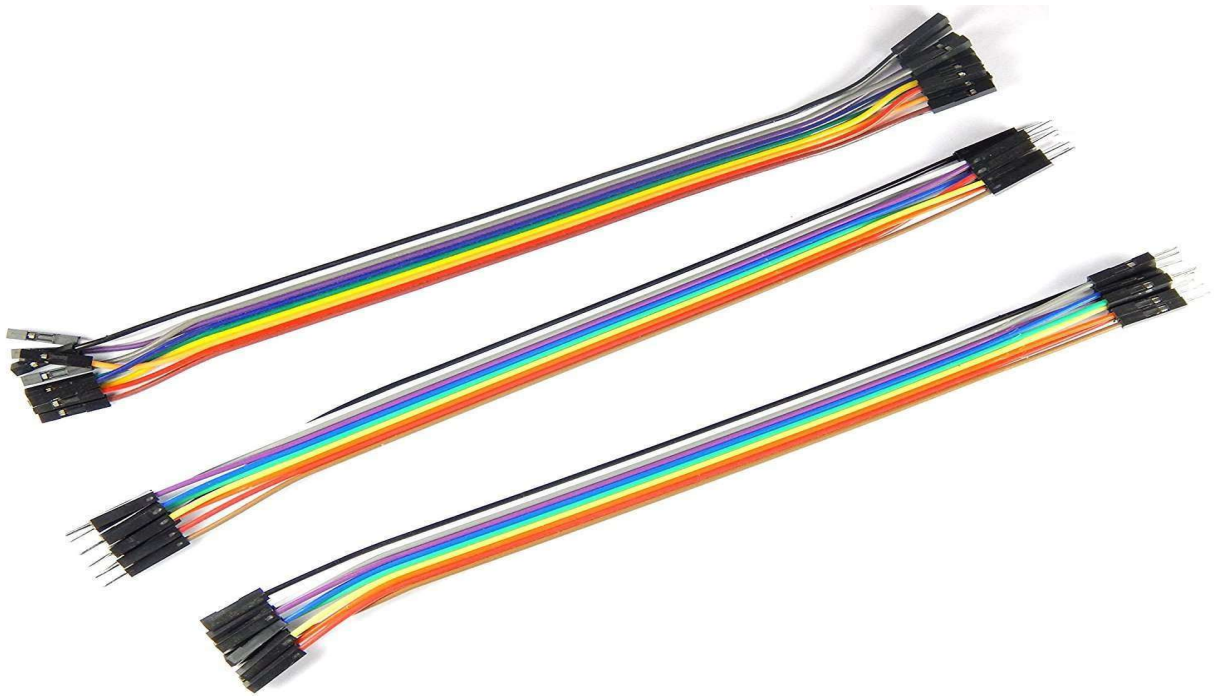


Fig. 4.6 Jumper Wires (Male to Male, Male to Female, Female to Female)

7. Breadboard

Used for assembling the prototype without soldering.

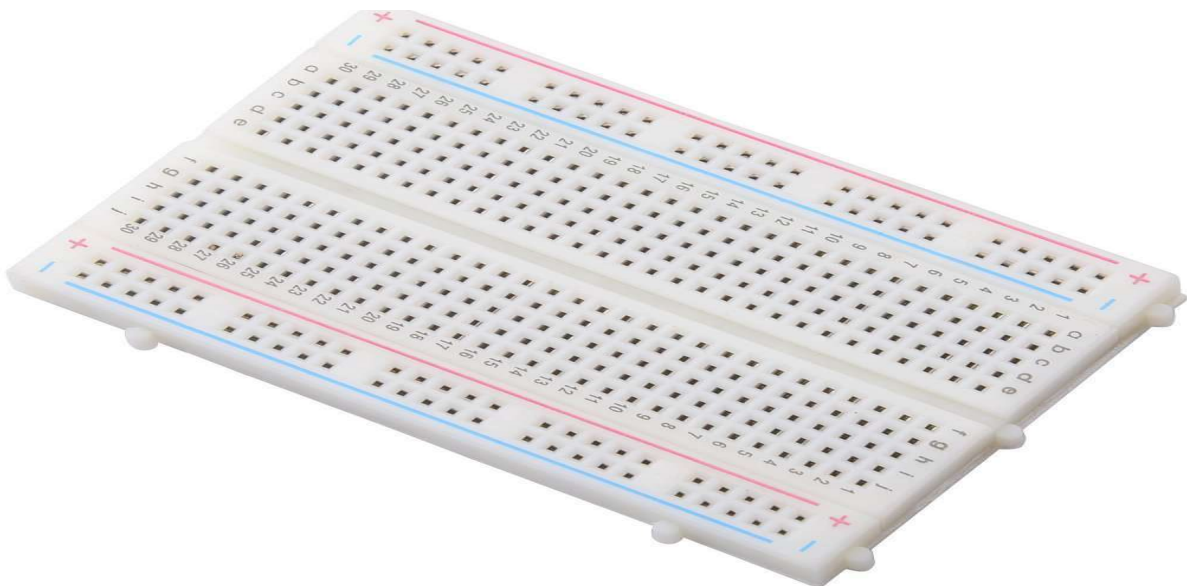


Fig. 4.7 Breadboard

CHAPTER – 5

CIRCUIT EXPLANATION

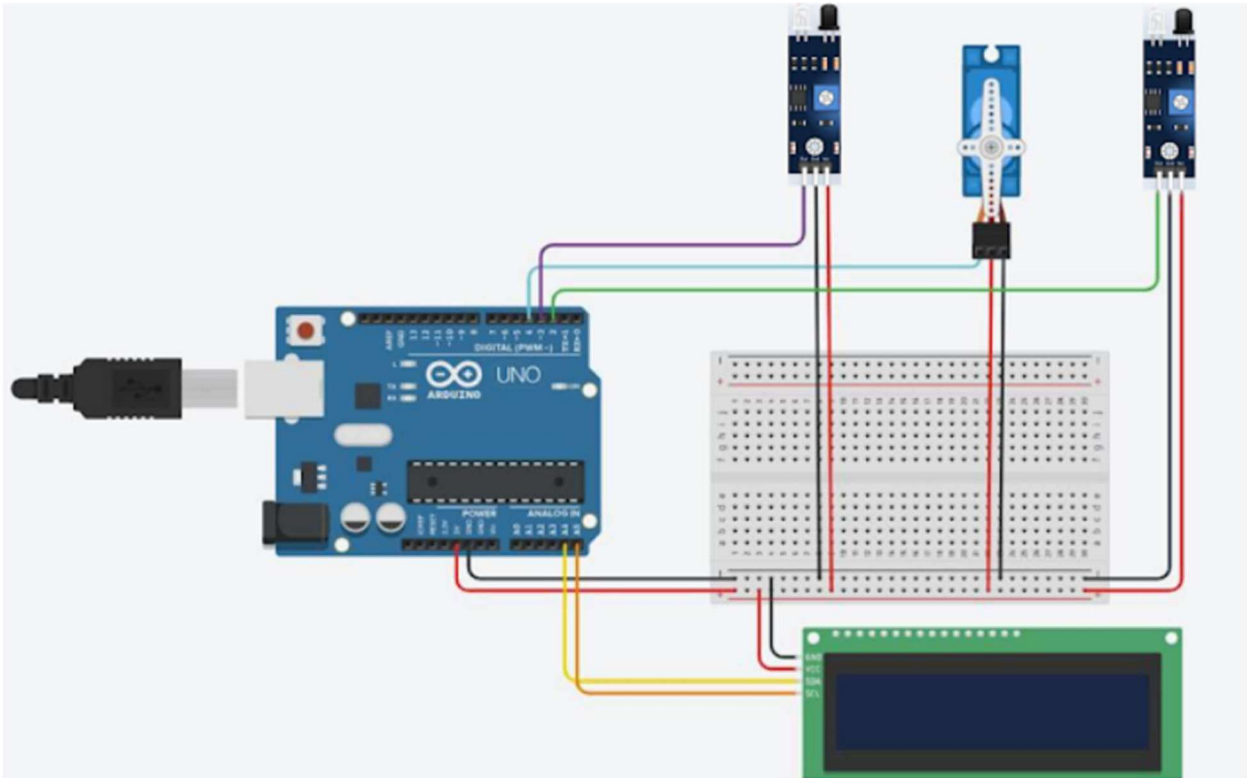


Fig. 5.1 Circuit Diagram

The Intelligent Urban Parking Space Optimization circuit is designed around Arduino Uno, which reads sensor inputs and controls the output devices. The system includes two IR sensors, a 16×2 LCD display with I2C module, and a servo motor acting as the gate barrier.

The circuit is assembled on a breadboard for easy prototyping. Each major part of the circuit is explained below:

1. IR Sensor Connections

Each IR sensor has three pins:

- VCC → 5V
- GND → GND
- OUT → Arduino Digital Pin (IR1 = D2, IR2 = D3)

When an object blocks the infrared beam, the output pin changes state (HIGH/LOW), depending on the module type.

Arduino monitors these changes to detect vehicle entry or exit.

2. Servo Motor Wiring

The SG90 servo motor has three wires:

- Red → 5V
- Brown/Black → GND
- Orange/Yellow (Signal) → Arduino Pin D9

Servo motors require stable 5V supply. The servo is controlled by PWM signals from Arduino, rotating the shaft to different angles (0°, 90°, 100° etc.).

In this project:

- 0° → Gate Open
- 100° → Gate Closed

3. LCD (16×2 with I2C)

The LCD module uses I2C interface to reduce wiring complexity.

Connections:

- GND → GND
- VCC → 5V
- SDA → A4
- SCL → A5

Using LiquidCrystal_I2C library, Arduino sends data to the display to show system status and remaining parking slots.

4. Arduino Uno as the Central Controller

Arduino Uno receives input signals from IR sensors and processes logic to:

- Update slot count
- Open/close servo gate
- Display messages on LCD

It acts as the decision-making unit.

5. Power Distribution

All modules share a common 5V and GND line.

A USB cable or 5V adapter powers the entire system.

6. Data Flow Summary

1. IR1 detects vehicle → Arduino decrements slot count
2. Arduino opens gate
3. IR2 detects exit → slot increments
4. LCD updated
5. Gate closes
6. Loop repeats

This makes the entire circuit automated and responsive.

CHAPTER – 6

WORKING OF THE SYSTEM

The Intelligent Urban Parking Space Optimization works on the principle of automatic sensing and monitoring.

Below is a detailed explanation:

Step 1: System Initialization

When power is supplied:

- Arduino starts running the program
- LCD shows welcome message
- Servo moves to closed position
- Initial slot count is displayed

Step 2: Vehicle Approaches Entry Gate

When a vehicle comes near IR Sensor 1:

- The IR beam is interrupted
- IR1 sends a HIGH or LOW signal to Arduino

If slots are available:

- Arduino reduces slot count
- Sends signal to servo
- Servo rotates to $0^\circ \rightarrow$ Gate Opens

If slots are full:

- LCD displays “SORRY — PARKING FULL”
- Servo stays closed

Step 3: Vehicle Moves Past Gate

When the vehicle leaves the entry area and passes IR2:

- IR2 detects the movement
- Arduino increments the slot count
- Servo rotates back to $100^\circ \rightarrow$ Gate Closed

Step 4: Updating the Display

LCD continuously shows:

```
WELCOME!  
Slot Left: X
```

Where X = remaining slots

Step 5: Continuous Monitoring

Arduino runs an infinite loop, constantly:

- Reading IR sensors
- Updating the slot counter
- Controlling servo gate
- Displaying information

The system operates smoothly and automatically without any manual control.

CHAPTER - 7

FLOWCHART

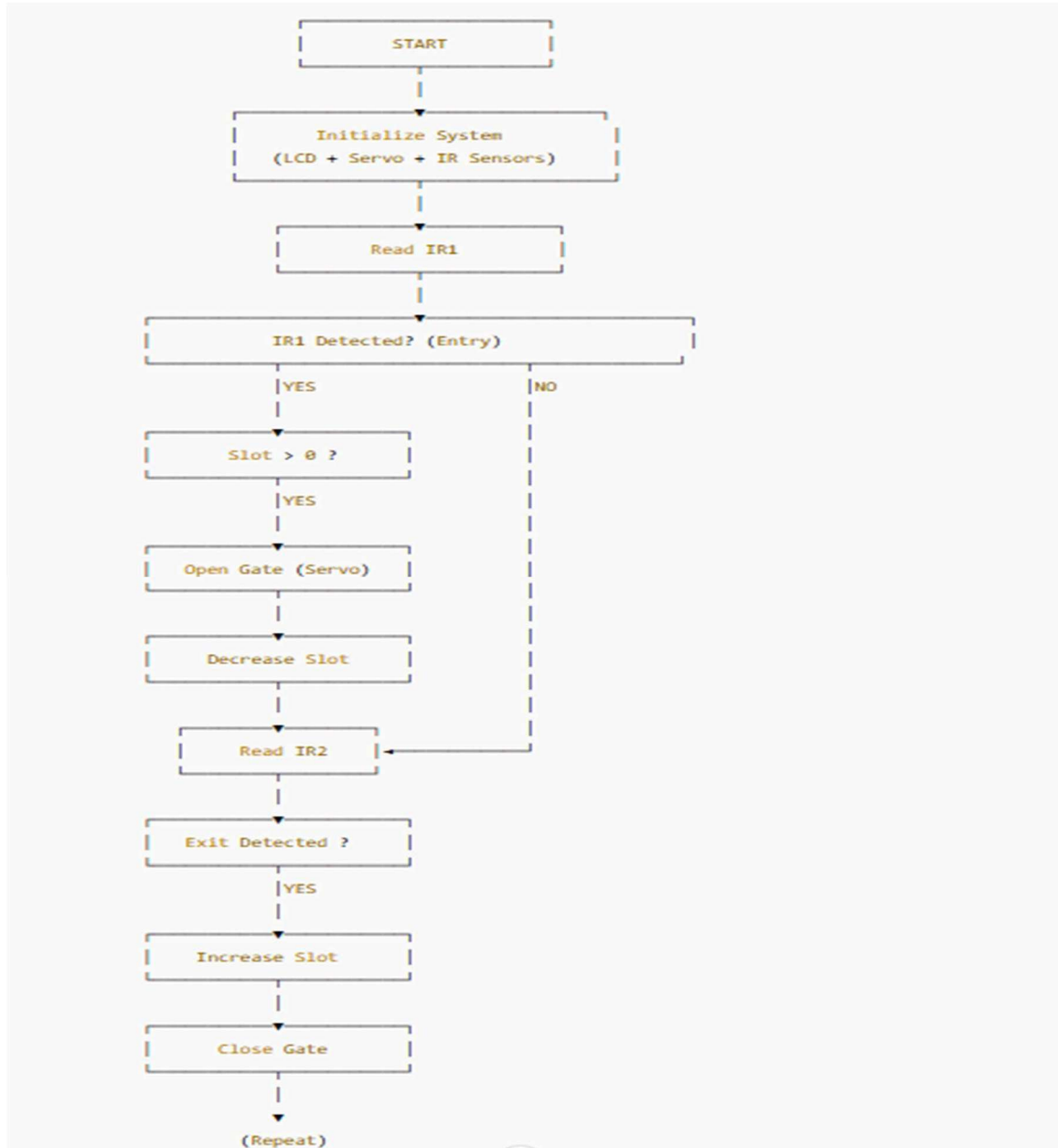


Fig. 7.1 Flowchart of the project

ALGORITHM

- Start the system.
- Initialize LCD, IR sensors, and servo motor.
- Set Slot = 4.
- Read IR1 input.
- If IR1 detects a vehicle and slot > 0:
 - Decrease Slot
 - Open gate (servo to 0°)
- If slot = 0:
 - Display “Parking Full”
 - Keep gate closed
- Read IR2 input.
- If IR2 detects a vehicle:
 - Increase Slot
 - Close gate (servo to 100°)
- Update LCD display.
- Repeat steps 4–9 forever.

CHAPTER – 8

SOFTWARE USED – Arduino IDE

Arduino IDE Features

- Code editor
- Serial Monitor
- Board manager
- Library manager
- Automatic compilation and upload

Libraries Required:

1. Wire.h – For I2C communication
2. LiquidCrystal_I2C.h – For LCD display
3. Servo.h – For servo motor control

Why Arduino IDE?

- Easy to write code
- Supports C++
- Thousands of examples
- Free and open-source

Uploading Steps

1. Connect Arduino to PC via USB
2. Open Arduino IDE
3. Select board → Arduino Uno
4. Select port (COM3/COM4 etc.)
5. Paste the code
6. Click Upload
7. Monitor output on Serial Monitor

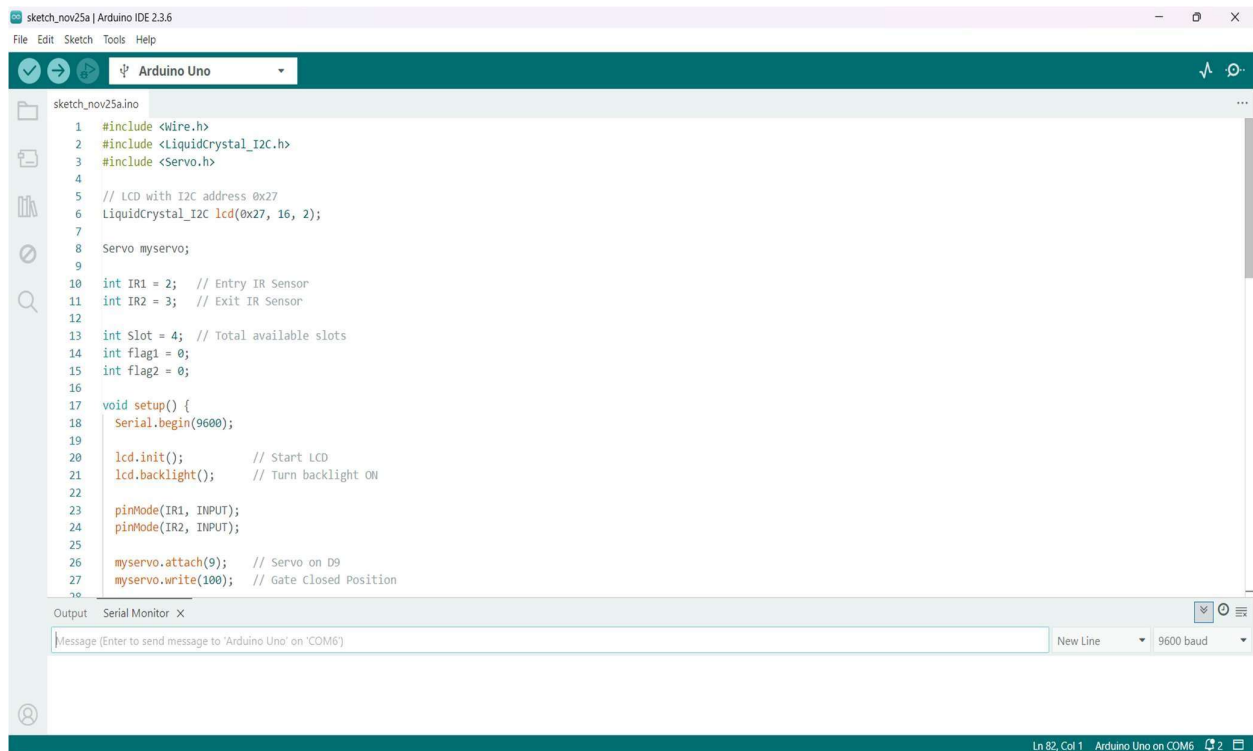


Fig. 8.1 Arduino IDE Workspace

ARDUINO CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>

// LCD with I2C address 0x27
LiquidCrystal_I2C lcd(0x27, 16, 2);

Servo myservo;

int IR1 = 2; // Entry IR Sensor
int IR2 = 3; // Exit IR Sensor

int Slot = 4; // Total available slots
int flag1 = 0;
int flag2 = 0;

void setup() {
  Serial.begin(9600);

  lcd.init();          // Start LCD
  lcd.backlight();      // Turn backlight ON

  pinMode(IR1, INPUT);
  pinMode(IR2, INPUT);

  myservo.attach(9);    // Servo on D9
  myservo.write(100);   // Gate Closed Position

  lcd.setCursor(0,0);
  lcd.print(" URBAN ");
  lcd.setCursor(0,1);
  lcd.print(" PARKING SYSTEM ");
  delay(2000);
  lcd.clear();
}

void loop() {

  // ENTRY
  if(digitalRead(IR1) == HIGH && flag1 == 0){
    if(Slot > 0){
      flag1 = 1;
      if(flag2 == 0){
        myservo.write(0);    // Open Gate
        Slot--;              // Reduce slot count
      }
    }
  }
}
```

```

    }
else{
    lcd.setCursor(0,0);
    lcd.print(" SORRY :( ");
    lcd.setCursor(0,1);
    lcd.print(" PARKING FULL ");
    delay(2500);
    lcd.clear();
}
}

// EXIT
if(digitalRead(IR2) == HIGH && flag2 == 0){
    flag2 = 1;
    if(flag1 == 0){
        myservo.write(0);    // Open Gate
        Slot++;             // Increase slot count
    }
}

// RESET FLAGS AND CLOSE GATE
if(flag1 == 1 && flag2 == 1){
    delay(1000);
    myservo.write(100);    // Close Gate
    flag1 = 0;
    flag2 = 0;
}

// LCD Display
lcd.setCursor(0,0);
lcd.print(" WELCOME! ");
lcd.setCursor(0,1);
lcd.print("Slot Left: ");
lcd.print(Slot);
}

```

CODE EXPLANATION

Header Files

- Wire.h enables I2C communication between Arduino and LCD.
- LiquidCrystal_I2C.h controls the 16×2 LCD display.
- Servo.h controls the servo motor.

Variable Declarations

- IR1 and IR2 store pin numbers connected to entry and exit IR sensors.
- Slot stores total available parking slots (initial value = 4).
- flag1 and flag2 act as detection flags to avoid duplicate counting.

Setup Function

- LCD initialized; backlight turned on.
- IR sensors configured as input pins.
- Servo attached to pin 9 and initialized to closed position (100°).
- Welcome message displayed on LCD.

Loop Function

It continuously performs:

1. Entry Detection (IR1)

- Checks if IR1 detects a vehicle.
- If slots are available → gate opens → decrement slot.
- If full → display “Parking Full”.

2. Exit Detection (IR2)

- Checks if a vehicle exits through IR2.
- Increments slot count.
- Opens gate briefly.

3. Gate Mechanism

After both sensors detect movement, servo:

- Delays 1 second
- Returns to closed position

Ensures that gate does not flap or move unnecessarily.

4. LCD Display Section

Continuously updates:

```
WELCOME!  
Slot Left: X
```

This gives drivers real-time slot availability.

CHAPTER - 9

ADVANTAGES OF THE SYSTEM

- Low cost — Uses inexpensive components like IR sensors and Arduino Uno.
- Real-time monitoring — LCD shows live parking status.
- Automated control — Gate opens/closes automatically.
- Reduced manpower — No guards needed to check available slots.
- Energy efficient — IR sensors consume low power.
- Accurate detection — Reliable vehicle detection using IR beams.
- Easy to build and maintain — Breadboard-based, modular design.
- Scalable — Can be expanded to more sensors or IoT-based cloud monitoring.
- Beginner friendly — Ideal for learning automation.

APPLICATIONS

This prototype can be deployed in various real-world scenarios such as:

1. Shopping malls

Automated entry gates controlling vehicle flow.

2. Residential apartments

Helps residents find free slots.

3. Corporate office buildings

Efficient parking management for employees.

4. Academic institutes

Smart campus-level parking systems.

5. Hospitals

To monitor emergency parking availability.

6. Event or Exhibition Halls

Manages high traffic efficiently.

7. Small business parking areas

Low-cost deployment for markets and shops.

LIMITATIONS OF THE SYSTEM

1. IR sensors can be affected by sunlight or reflective surfaces.
2. Limited range — works only for short distance detection.
3. Not ideal for high-speed vehicles.
4. Cannot detect vehicle size or type.
5. Servo motor cannot lift heavy barriers.
6. Slot management is basic — not suitable for large parking lots.
7. No online monitoring (only LCD display).

These limitations can be overcome using ultrasonic sensors, stepper motors, or IoT modules.

CHAPTER - 10

RESULTS & OBSERVATIONS

Results

- System successfully detects entry and exit using IR sensors.
- Gate opens/closes correctly through the servo motor.
- LCD updates parking slot count in real-time.
- System responds quickly with no noticeable lag.
- Fully functional working prototype achieved.

Observations

- Servo performance improves with stable 5V supply.
- Sensitivity adjustment of IR sensors is crucial.
- Arduino processes the logic accurately without missing signals.
- Breadboard connections must be tight to avoid false triggers.

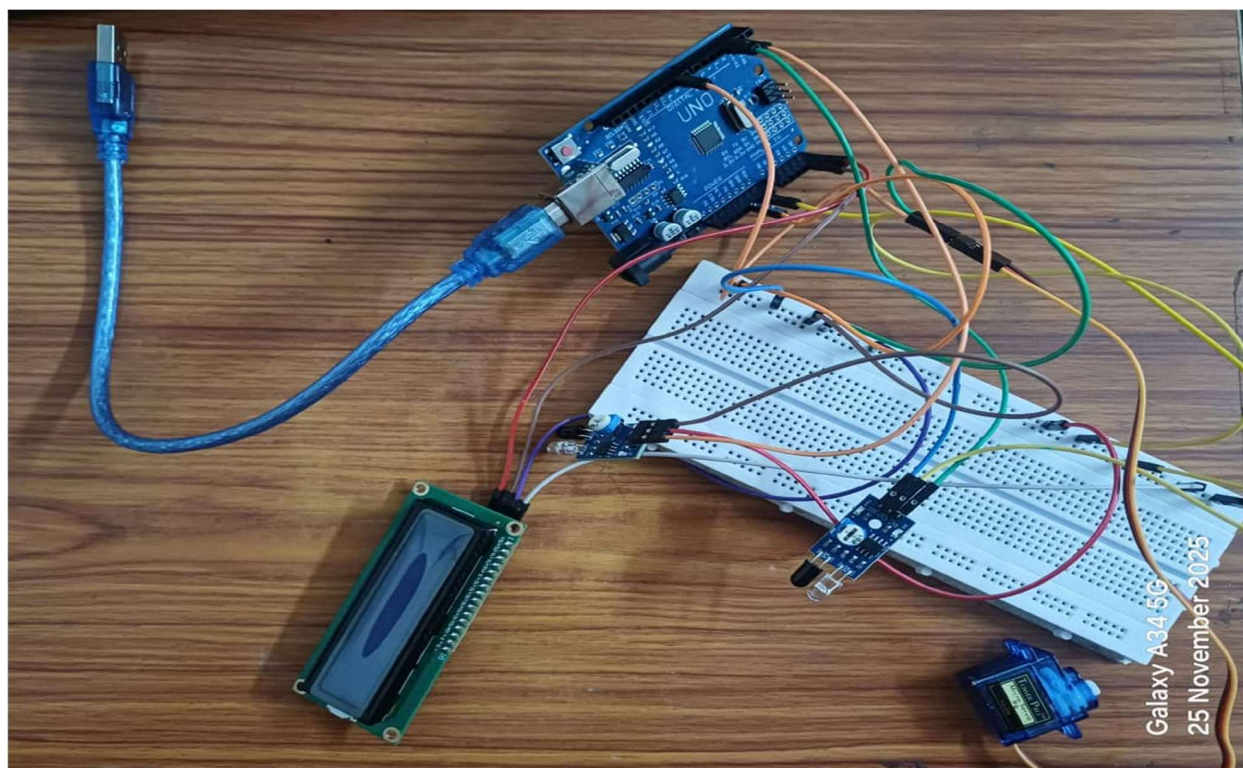
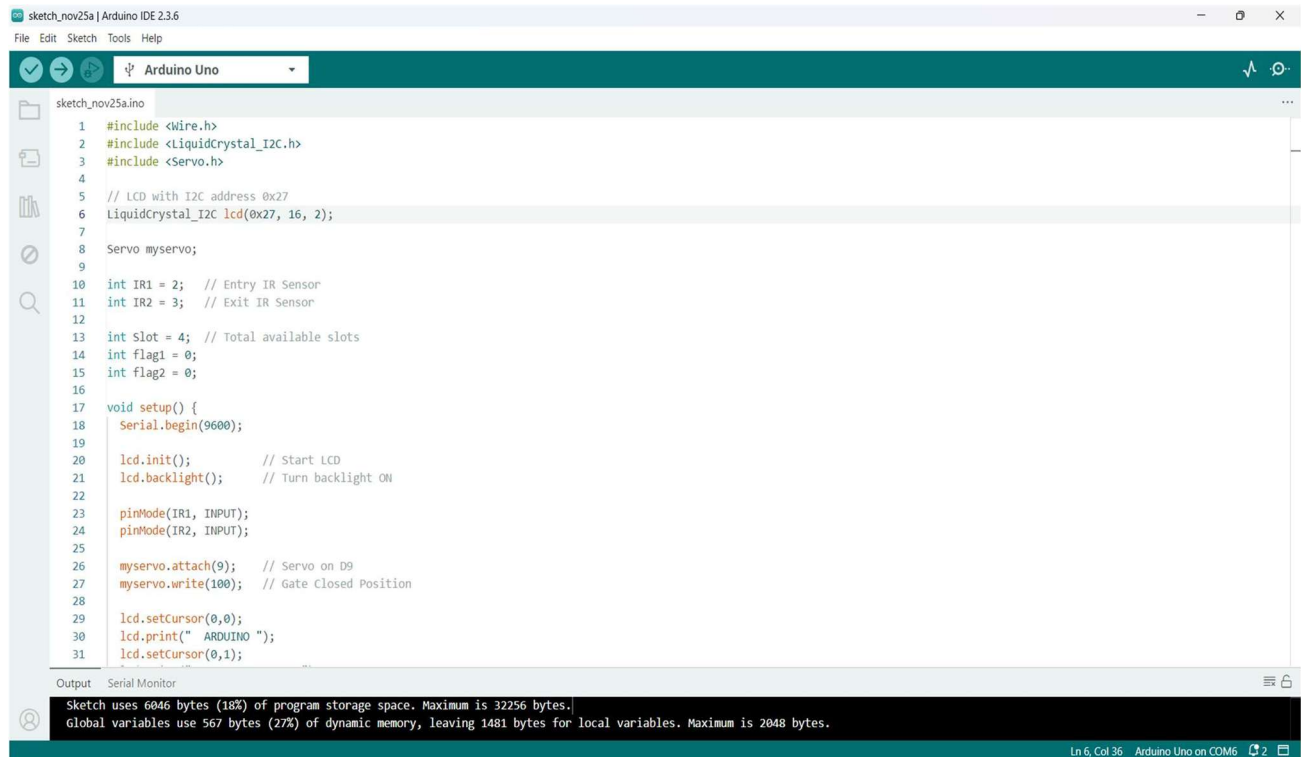


Fig. 10.1 Circuit Implementation



Fig. 10.2 LCD display successful



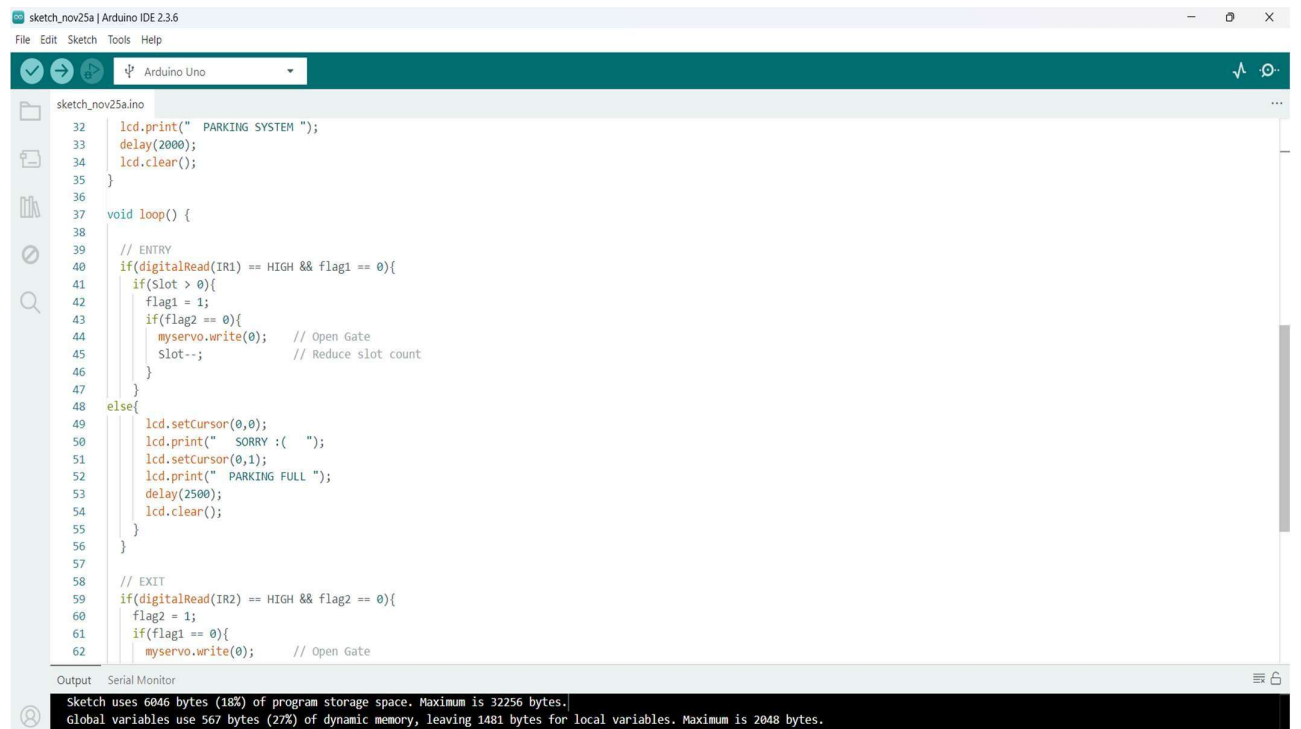
```
1 #include <Wire.h>
2 #include <LiquidCrystal_I2C.h>
3 #include <Servo.h>
4
5 // LCD with I2C address 0x27
6 LiquidCrystal_I2C lcd(0x27, 16, 2);
7
8 Servo myservo;
9
10 int IR1 = 2; // Entry IR Sensor
11 int IR2 = 3; // Exit IR Sensor
12
13 int Slot = 4; // Total available slots
14 int flag1 = 0;
15 int flag2 = 0;
16
17 void setup() {
18   Serial.begin(9600);
19
20   lcd.init(); // Start LCD
21   lcd.backlight(); // Turn backlight ON
22
23   pinMode(IR1, INPUT);
24   pinMode(IR2, INPUT);
25
26   myservo.attach(9); // Servo on D9
27   myservo.write(100); // Gate Closed Position
28
29   lcd.setCursor(0,0);
30   lcd.print(" ARDUINO ");
31   lcd.setCursor(0,1);
32 }
```

Output Serial Monitor

Sketch uses 6046 bytes (18%) of program storage space. Maximum is 32256 bytes.
Global variables use 567 bytes (27%) of dynamic memory, leaving 1481 bytes for local variables. Maximum is 2048 bytes.

Ln 6, Col 36 Arduino Uno on COM6

Fig. 10.3(a) Arduino Code Successful



```
32   lcd.print(" PARKING SYSTEM ");
33   delay(2000);
34   lcd.clear();
35 }
36
37 void loop() {
38
39   // ENTRY
40   if(digitalRead(IR1) == HIGH && flag1 == 0){
41     if(Slot > 0){
42       flag1 = 1;
43       if(flag2 == 0){
44         myservo.write(0); // Open Gate
45         Slot--; // Reduce slot count
46       }
47     }
48   } else{
49     lcd.setCursor(0,0);
50     lcd.print(" SORRY :( ");
51     lcd.setCursor(0,1);
52     lcd.print(" PARKING FULL ");
53     delay(2500);
54     lcd.clear();
55   }
56 }
57
58 // EXIT
59 if(digitalRead(IR2) == HIGH && flag2 == 0){
60   flag2 = 1;
61   if(flag1 == 0){
62     myservo.write(0); // Open Gate
63 }
```

Output Serial Monitor

Sketch uses 6046 bytes (18%) of program storage space. Maximum is 32256 bytes.
Global variables use 567 bytes (27%) of dynamic memory, leaving 1481 bytes for local variables. Maximum is 2048 bytes.

Fig. 10.3(b) Arduino Code Successful

```
63     slot++;          // Increase slot count
64   }
65 }
66
67 // RESET FLAGS AND CLOSE GATE
68 if(flag1 == 1 && flag2 == 1){
69   delay(1000);
70   myservo.write(100); // Close Gate
71   flag1 = 0;
72   flag2 = 0;
73 }
74
75 // LCD Display
76 lcd.setCursor(0,0);
77 lcd.print("  WELCOME!  ");
78 lcd.setCursor(0,1);
79 lcd.print("Slot Left: ");
80 lcd.print(slot);
81 }
82
```

Output Serial Monitor

Sketch uses 6046 bytes (18%) of program storage space. Maximum is 32256 bytes.
Global variables use 567 bytes (27%) of dynamic memory, leaving 1481 bytes for local variables. Maximum is 2048 bytes.

Fig. 10.3(c) Arduino Code Successful

TESTING & ANALYSIS

Test Case 1: Vehicle Approaches IR1

Output: Slot decreases, gate opens

Result: Success

Test Case 2: Parking Full

Output: Gate does not open

Result: Success

Test Case 3: Vehicle Exits (IR2)

Output: Slot increases

Result: Success

Test Case 4: Rapid Movement

Output: Flags prevent duplicate counting

Result: Success

CHAPTER – 11

FUTURE SCOPE

Intelligent Urban Parking Space Optimization are an important part of modern Smart Cities, and this project can be improved greatly. Below are major future scope points:

1. IoT Integration (Internet of Things)

- ESP8266/ESP32 modules can upload slot data to the cloud.
- User can check slots on a mobile app before arriving.
- Real-time online monitoring.

2. Android App for Slot Booking

- Users can reserve a parking slot from their phone.
- Avoids unnecessary vehicle movement.

3. Ultrasonic Sensors instead of IR

- More accurate distance measurement.
- Less affected by sunlight.

4. Automatic Payment System

- RFID cards for monthly members
- QR code scanning
- Pay-per-hour parking

5. ANPR Camera Integration

- Automatic Number Plate Recognition
- Logs vehicle entry with timestamp

6. Large-Scale Deployment

- Multi-floor parking
- Multi-row parking
- Smart lighting and guidance systems

7. Solar Power Supply

- Convert the system into eco-friendly solar-powered automation.

CHAPTER – 12

CONCLUSION

The Intelligent Urban Parking Space Optimization using Arduino successfully demonstrates how automation can simplify everyday problems. By using only two IR sensors, a servo motor, an LCD display, and Arduino Uno, the system provides real-time slot availability and automatic gate control.

The project fulfils all its objectives:

- Detect vehicle entry and exit accurately
- Display available slots
- Automate barrier gate operation
- Provide a user-friendly interface

This working prototype proves that even with low-cost components, reliable and efficient embedded solutions can be developed. The project is scalable and can be upgraded into a fully IoT-enabled smart parking system suitable for real-world deployment.

It is a perfect example of how electronics engineering, embedded systems, and automation work together to create practical solutions.

CHAPTER – 13

REFERENCES

- [1] A. T. Al-Humaidi and M. A. Mahmood, “Design and Implementation of Smart Parking System Using Arduino,” *International Journal of Engineering Research & Technology*, vol. 8, no. 4, pp. 50–56, 2021.
- [2] S. R. Bharamagoudar, R. R. Patil, and M. S. Sasnur, “IoT-Based Smart Parking System for Smart Cities,” in *Proc. Int. Conf. Smart City Applications*, 2019, pp. 1–6.
- [3] A. K. Singh and P. Gupta, “Real-Time Vehicle Detection Using Infrared Sensors,” *IEEE Sensors Journal*, vol. 18, no. 14, pp. 5765–5772, 2019.
- [4] M. R. Hossain, “Microcontroller-Based Parking Control System,” in *IEEE Int. Conf. Electrical, Computer and Communication Engineering (ICECCE)*, Dhaka, Bangladesh, 2021, pp. 182–186.
- [5] J. Smith and K. Lee, “Low-Cost Embedded Systems for Smart Parking Infrastructure,” *IEEE Access*, vol. 9, pp. 110233–110245, 2021.
- [6] T. Ahmad, P. Roy, and R. Das, “Smart Parking System Using Sensor Networks and Embedded Microcontrollers,” in *IEEE Int. Conf. Intelligent Transportation Engineering*, 2020, pp. 112–118.
- [7] Arduino, “Arduino Uno Rev3 Technical Documentation,” Arduino.cc, 2024. [Online]. Available: <https://www.arduino.cc>